

Step 5 — Rule Engine (Digital Twin)

Goal

Every device follows engineering rules.

Instead of:

CPU = Random(20,80)

we'll have:

CPU = f(FPS, Resolution, Bitrate)

Every property is calculated.

Core Philosophy

Every parameter belongs to one of three categories.

1. Configurable Properties (User changes)

Examples:

- Resolution
- FPS
- Codec
- Bitrate
- Number of Streams
- Fan Speed

These are editable.

2. Calculated Properties

Examples:

- CPU
- Memory
- Power
- Temperature

- Network Load

These are **never edited directly**.

They are calculated.

3. State Properties

Examples:

- ONLINE
- WARNING
- FAILED
- RECOVERING

These are determined automatically.

Property Model (Improved)

Instead of primitive fields:

fps = 60;

Every property becomes an object.

Property

name

value

unit

editable

minimum

maximum

category

Example

FPS

60

fps

editable=tru

min=1

max=240

This makes the property panel dynamic.

Rule Graph

Every device has a dependency graph.

Example Encoder

Resolution



Bitrate



CPU



Power



Temperature



Health

Instead of hardcoding calculations everywhere, each device owns its dependency graph.

Device Rule Model

Every device implements

calculate()

Example

Camera



calculate()



updateBandwidth()



updatePower()



updateTemperature()

The Simulation Engine doesn't know the formulas—it just tells each device to recalculate.

Rule Chains

Example

User changes

FPS

30 → 60

Rule Engine

Bandwidth +30%



CPU +18%



Power +12%



Temperature +8°C



Health = WARNING

This creates natural cause-and-effect.

Threshold System

Each property may have thresholds.

Example Temperature

0–70°C

NORMAL

70–85°C

WARNING

85°C+

CRITICAL

CPU

0–80%

NORMAL

80–95%

WARNING

95%+

CRITICAL

The Alarm Engine simply evaluates thresholds.

Formula Profiles

Instead of putting formulas inside the engine, each device owns a **Formula Profile**.

Example

Encoder

CPU

=

BaseCPU

+

ResolutionCost

+

FPSCost

+

CodecCost

Router

CPU

=

BaseCPU

+

Streams

+

Bandwidth

Camera

Power

=

BasePower

+

FPS

+

Resolution

Each device behaves differently.

Rule Categories

Performance Rules

- CPU
 - Memory
-

Thermal Rules

- Power
 - Temperature
 - Cooling
-

Network Rules

- Bandwidth
 - Packet Loss
 - Latency
-

Health Rules

- Warning
 - Failure
 - Recovery
-

Property Dependencies

Instead of coding

if FPS changes...

We'll maintain a dependency map.

Example

FPS



Bandwidth



CPU



Power



Temperature

The Rule Engine traverses this graph.

Property Update Flow

User changes FPS



Mark FPS as Changed



Rule Engine finds dependencies



Recalculate Bandwidth



Recalculate CPU



Recalculate Temperature



Notify Alarm Engine

Only affected properties are recalculated.

Rule Registry ★

Every device registers its own rules.

Camera Rules

Encoder Rules

Router Rules

Media Server Rules

The Simulation Engine doesn't know formulas.

It simply asks:

Device

↓

calculate()

This follows good object-oriented design.

Engineering Constants ★

Instead of magic numbers,

create constants.

Example

1080p Cost

4K Cost

H264 Cost

H265 Cost

Temperature Warning

CPU Warning

Easy to tune later.

Formula Complexity

Keep formulas simple but believable.

Example

$\text{CPU} = \text{Base} + \text{ResolutionFactor} + \text{FPSFactor} + \text{CodecFactor}$

Temperature

$\text{Temperature} += \text{Power} \times \text{CoolingFactor}$

Bandwidth

$\text{Bandwidth} = \text{Resolution} \times \text{FPS} \times \text{CompressionFactor}$

These don't need to be physically exact—they need to be **consistent** and demonstrate relationships.

Future Extension

In Version 2, users could even define custom rules:

If Temperature > 90

↓

Reduce FPS automatically

Or:

If CPU > 95%

↓

Throttle Bitrate

This would turn the simulator into a configurable engineering platform.

Final Rule Engine Architecture

Rule Engine

|

├— Property Manager

├— Dependency Graph

├— Formula Registry

├— Threshold Evaluator

└─ Rule Executor

└─ Change Tracker

Important Design Decision

Device-Specific Rules

Rather than one giant RuleEngine class with lots of if (deviceType == ENCODER) checks, every device should encapsulate its own engineering model.

For example:

AbstractDevice

```
|
|
|   └─ Camera
|       └─ CameraRuleModel
|
|
|   └─ Encoder
|       └─ EncoderRuleModel
|
|
|   └─ Router
|       └─ RouterRuleModel
```

The central Rule Engine simply coordinates execution; it doesn't contain device-specific knowledge.

At this point, our architecture is about 70% complete.

We've already defined:

- ☒ Overall Architecture
- ☒ Device Framework
- ☒ Simulation Engine
- ☒ Rule Engine